



PRAXISORIENTIERTE FALLSTUDIE (AI3955)

Sommersemester 2026

Verfügbarkeit und Fehlertoleranz des ROSI-Systems

im industriellen Dauerbetrieb

vorgelegt von:

Luca Michael Schmidt (Matr.-Nr. 1540963)

Hochschulbetreuer:

Prof. Dr. Thomas Wiemann

Betreuer im Unternehmen:

Manuel Pilz (Grenzebach BSH GmbH)

Hochschule Fulda – Fachbereich Angewandte Informatik

24. Juli 2026

Sperrvermerk

Die vorliegende Arbeit beinhaltet interne und vertrauliche Informationen der Grenzebach BSH GmbH. Die Weitergabe des Inhalts der Arbeit sowie der darin enthaltenen Daten ist im Ganzen oder in Teilen grundsätzlich untersagt. Es dürfen keine Kopien oder Abschriften, auch nicht in digitaler Form, gefertigt werden. Ausnahmen bedürfen der schriftlichen Genehmigung der Grenzebach BSH GmbH.

Zusammenfassung

Diese praxisorientierte Fallstudie analysiert die Verfügbarkeit und Fehlertoleranz des ROSI-Systems (*Realtime Optical Surface Inspection*) im industriellen 24/7-Betrieb. ROSI ist ein On-Premise-Inline-Scan-System der Grenzebach BSH GmbH zur automatisierten Qualitätskontrolle von Baustoffen und technisch als Python-Multiprocessing-System mit Shared-Memory-basierter Verarbeitungspipeline realisiert.

Der Schwerpunkt liegt auf Fehlern, die im Dauerbetrieb schleichend entstehen, lange unbeachtet bleiben oder kaskadieren und im Entwicklungsbetrieb daher selten sichtbar werden. Fünf Failure Cases auf Prozess- und Betriebssystemebene wurden in einer kontrollierten Testumgebung mittels Fault Injection reproduzierbar untersucht.

Die Messungen zeigen konkrete Lücken in der Fehlertoleranz: Drei von vier Service-Prozessen fallen ohne Erkennung aus; eine volle RAM-Disk führt nach 35 s Stillstand zum Absturz der Anlage; eine blockierende Log-Queue hält die Pipeline ohne Alarm an. Eine veraltete Datenbankverbindung bleibt in der Entwicklungsumgebung unsichtbar, ist in der Produktionskonfiguration aber als Risiko ableitbar. Eine angenommene Race Condition im Speichermanagement wurde widerlegt. Aus den Befunden werden aufwandsarme Maßnahmen zur besseren Überwachung und Fehlerentkopplung abgeleitet.

Inhaltsverzeichnis

1	Einleitung und Problemstellung	1
1.1	Problemstellung	1
1.2	Zielsetzung und Forschungsfragen	2
1.3	Aufbau der Arbeit	2
2	Systemarchitektur von ROSI	2
2.1	Prozess-Topologie	3
2.2	Verarbeitungspipeline und Worker-Pool	3
2.3	Speichermanagement: Shared Memory und Zero-Copy	5
3	Theoretische Grundlagen	5
3.1	Verfügbarkeit im Dauerbetrieb	5
3.2	Fehlerklassifikation	6
3.3	Prozesssupervision und das Watchdog-Pattern	6
3.4	Ressourcenerschöpfung und Backpressure	7
3.5	Dev/Prod-Parity und Konfigurationsgefälle	7
4	Methodik	7
4.1	Untersuchungsansatz	7
4.2	Verfahren der Fehlerinjektion	8
4.3	Testumgebung	8
4.4	Referenzmessung (Baseline)	9
4.5	Messgrößen	9
4.6	Grenzen der Untersuchung	9
5	Failure Cases	10
5.1	Case 1 – Lücken in der Laufzeit-Prozesssupervision	10
5.2	Case 2 – Flüchtiger Bildspeicher als Single Point of Failure	10
5.3	Case 3 – Log-Queue-Stau als Pipeline-Blocker	12
5.4	Case 4 – Veraltete Datenbankverbindung nach Produktionspause	13
5.5	Case 5 – Vermutete Race Condition im Speichermanagement	13
6	Fazit und Handlungsempfehlungen	14
6.1	Übergreifende Befunde	14
6.2	Priorisierte Handlungsempfehlungen	15
6.3	Ausblick	15
	Literaturverzeichnis	16

1 Einleitung und Problemstellung

ROSI (*Realtime Optical Surface Inspection*) ist ein von der Grenzebach BSH GmbH entwickeltes Inline-Scan-System zur automatisierten Qualitätskontrolle von Baustoffen. Im laufenden Produktionsprozess werden Materialoberflächen erfasst, Defekte per KI-Inferenz erkannt und das Material anschließend regelbasiert in Qualitätsstufen klassifiziert. Der Betrieb erfolgt On-Premise direkt in der Produktionsanlage des Kunden im 24/7-Dauerbetrieb.

Technisch ist ROSI als Python-Multiprocessing-System aufgebaut. Mehrere langlebige Prozesse verarbeiten die Daten in einer Shared-Memory-basierten Pipeline, ergänzt durch Queues und flüchtigen Zwischenspeicher in Form eines tmpfs-RAM-Dateisystems. Große Bilddaten wandern dabei nicht durch Queues, sondern verbleiben in vom Betriebssystem verwalteten Speicherblöcken; die Queues transportieren ausschließlich Referenzen. Die in dieser Fallstudie untersuchten Fehlerklassen setzen genau an dieser Prozess- und Speicherarchitektur an.

ROSI befindet sich in aktiver Entwicklung; die erste Kundeninbetriebnahme ist für Mitte bis Ende 2026 geplant. Mit dem bevorstehenden Produktiveinsatz steigen die Anforderungen an Verfügbarkeit und Fehlertoleranz: Ausfälle können Produktionsstillstände verursachen, Inspektionsergebnisse verhindern und im ungünstigsten Fall Qualitätsmängel unentdeckt lassen.

1.1 Problemstellung

Systeme im 24/7-Betrieb sind Fehlerklassen ausgesetzt, die im Entwicklungsbetrieb unsichtbar bleiben. Ein System, das täglich neu gestartet wird und unter manueller Beobachtung steht, durchläuft selten jene Zustände, in denen sich Ressourcen schleichend erschöpfen, Verbindungen veralten oder Teilprozesse still ausfallen. Für ROSI lassen sich die relevanten Risikokategorien wie folgt einteilen:

- *Speichererschöpfung*: Puffer, Logs oder Datenbanken wachsen schleichend, bis Schreiboperationen systemweit fehlschlagen.
- *Stille Prozessausfälle*: Ein Teilprozess stürzt ab, das System läuft scheinbar weiter – ohne Alarm und ohne Neustart.
- *Verbindungsabbrüche*: Datenbankverbindungen veralten nach Produktionspausen und führen beim nächsten Zugriff zu unkontrollierten Prozessabstürzen.
- *Kaskadeneffekte*: Die Verlangsamung einer Komponente blockiert über gemeinsame Queues und Shared-Memory-Schnittstellen das gesamte System.
- *Blinde Flecken im Monitoring*: Vorhandene Überwachungskomponenten erfassen nicht alle relevanten Ressourcen, etwa flüchtige RAM-Dateisysteme oder interne Queue-Tiefen.

ROSI verfügt aktuell über kein externes Alerting. Probleme werden erst durch ausbleibende Inspektionsergebnisse oder manuelle Beobachtung sichtbar.

1.2 Zielsetzung und Forschungsfragen

Ziel dieser Fallstudie ist es, verfügbarkeitskritische Fehlerszenarien im industriellen Dauerbetrieb auf Prozess- und Betriebssystemebene zu identifizieren, reproduzierbar zu simulieren und anhand messbarer Kriterien systematisch zu bewerten. Im Fokus stehen stille, schleichende und kaskadierende Fehler. Die untersuchten Failure Cases werden in drei Themenblöcke gegliedert, denen jeweils eine Forschungsfrage (FF) zugeordnet ist:

- *FF1 – Speicher- und Ressourcenverwaltung:* Wie verhält sich das System, wenn flüchtige Speichersubsysteme überlaufen oder Timing-Konflikte im Shared-Memory-Management auftreten?
- *FF2 – Systembeobachtbarkeit und Fehlertoleranz:* In welchem Zeitraum erkennt das System eigene Prozessausfälle, und welche Reaktion erfolgt?
- *FF3 – Datenpersistenz und Kaskadeneffekte:* Wie verhält sich das System bei unterbrochenen Datenbankverbindungen oder blockierenden Queues, und welche Folgeausfälle entstehen?

1.3 Aufbau der Arbeit

Kapitel 2 beschreibt die für die Fehleranalyse relevante Prozess- und Speicherarchitektur von ROSI. Kapitel 3 legt den theoretischen Rahmen zu Verfügbarkeit, Fehlerklassifikation, Prozesssupervision und Ressourcenmanagement. Kapitel 4 beschreibt die Testumgebung und die Methodik der Fehlerinjektion. Kapitel 5 stellt die fünf Failure Cases mit ihren Messergebnissen dar. Kapitel 6 fasst die Befunde zusammen und leitet Handlungsempfehlungen ab.

2 Systemarchitektur von ROSI

Die untersuchten Fehlerklassen lassen sich erst mit Kenntnis der konkreten Prozess- und Speicherarchitektur einordnen. Dieser Abschnitt beschreibt die für die Failure Cases relevanten Bausteine: die Prozess-Topologie, die Verarbeitungspipeline mit ihrem Worker-Pool und das Shared-Memory-basierte Speichermanagement.

2.1 Prozess-Topologie

ROSI ist bewusst als Multiprocessing-System ausgelegt, um den *Global Interpreter Lock* von Python zu umgehen und große Bilddatenmengen nicht durch teure Inter-Prozess-Kommunikation zu blockieren. Der Startprozess `main.py` teilt das System in vier langlebige Service-Prozesse auf, die über threadsichere Queues und Event-Primitive kommunizieren (Abbildung 1). Der `InspectionDataHandler` erzeugt fortlaufend leere Daten-Container und schiebt sie über eine begrenzte Queue in das System; ist die Queue voll, blockiert er und wirkt so als systemweites Backpressure-Ventil gegen unbegrenzte Speicherallokation. Der `FrameGrabber` bindet die Kamerahardware an, schreibt die rohen Pixeldaten in Shared-Memory-Blöcke und reicht Referenzen darauf an den `PipelineManager` weiter. Dieser orchestriert die eigentliche Verarbeitung. Der `LoggerProcess` nimmt die Lognachrichten aller Prozesse über eine zentrale Queue entgegen und schreibt sie asynchron nach PostgreSQL, sodass I/O-Latenzen die Bildverarbeitung nicht ausbremsen.

Für die Fehlertoleranz ist eine Eigenschaft dieser Topologie zentral: `main.py` überwacht zur Laufzeit ausschließlich den `PipelineManager` über einen Poll im Sekundentakt. Die drei übrigen Service-Prozesse laufen nach erfolgreichem Start ohne jede Laufzeit-Supervision. Genau an dieser Lücke setzt Case 1 (Abschnitt 5.1) an; in Abbildung 1 sind die unüberwachten Prozesse rot markiert.

2.2 Verarbeitungspipeline und Worker-Pool

Der `PipelineManager` führt jede Inspektion durch eine streng sequenzielle Pipeline aus sechs Schritten. Rechenintensive Schritte – KI-Inferenz, Geometrievermessung und Bildkompression – lagert er über ein Map-Reduce-Muster in den `MPWorkerPool` mit zwölf Worker-Prozessen aus: Ein Schritt wird in Teiljobs zerlegt (Map), auf freie Worker verteilt und das Ergebnis wieder zusammengeführt (Reduce). Ein Scheduler wählt dabei den Worker mit der kürzesten Job-Queue. Zwei Hintergrund-Threads ergänzen den Pool: Der `ResultCollector` sammelt die Teilergebnisse nicht blockierend ein und benachrichtigt die Pipeline erst, wenn alle Teiljobs abgeschlossen sind; ein `HealthMonitor` überwacht die Heartbeats der Worker und startet einen nicht mehr antwortenden Worker neu. Diese Worker-Überwachung ist damit deutlich engmaschiger als die Supervision der Service-Prozesse – ein Gefälle, das für die Fehlerbilder in Kapitel 5 bedeutsam ist.

Tabelle 1 fasst die sechs Schritte mit ihren im Referenzlauf gemessenen Laufzeiten zusammen. Die kurzen, planbaren Schrittdauern sind später die Grundlage der Timing-Analyse in Case 5 (Abschnitt 5.5).

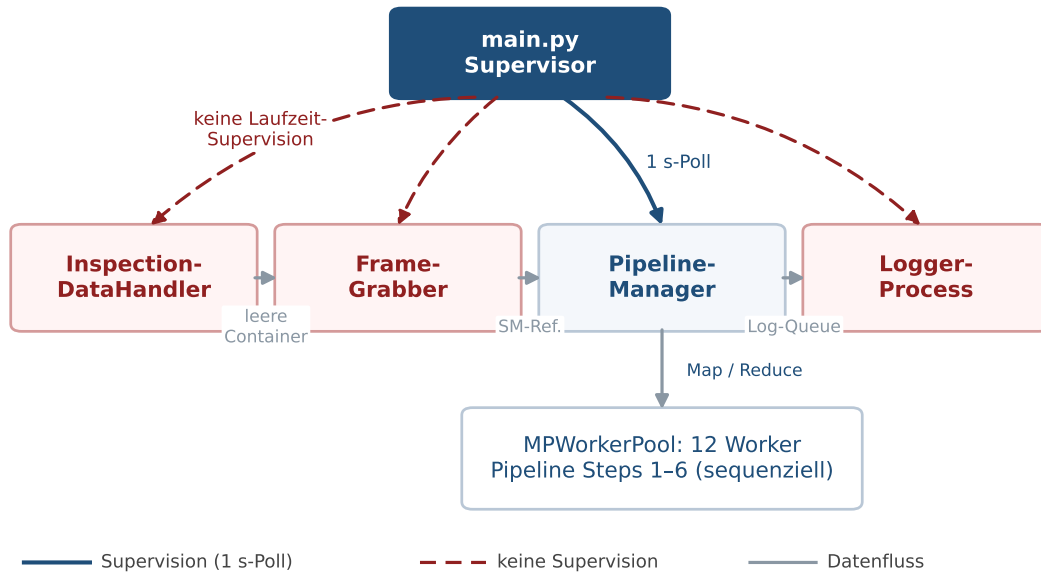


Abbildung 1: Prozess-Topologie von ROSI. `main.py` überwacht zur Laufzeit nur den `PipelineManager` (1 s-Poll); die drei rot markierten Service-Prozesse laufen ohne Laufzeit-Supervision (Case 1). Graue Pfeile zeigen den Datenfluss; große Bilddaten wandern nicht durch die Queues, sondern als Shared-Memory-Referenzen. Alle Prozesse loggen über die gemeinsame Log-Queue.

Quelle: Eigene Darstellung

Tabelle 1: Schritte der Verarbeitungspipeline mit Referenz-Laufzeiten

Step	Ausführung	Aufgabe	Ø Dauer
1	lokal	Warten auf neue Inspektionsdaten (<code>wait_for_new_inspection</code>)	10–20 ms
2	lokal	Geometrievermessung	25–70 ms
3	lokal	Zuschnitt der Region of Interest	5–15 ms
4	parallel	Defekterkennung (klassisch + KI) und Bildkodierung	170–220 ms
5	parallel	Merkmalsextraktion und Bildspeicherung	50–110 ms
6	lokal	Persistierung nach PostgreSQL (Django-ORM)	5–110 ms

2.3 Speichermanagement: Shared Memory und Zero-Copy

Das kritischste Entwurfsmuster von ROSI ist die Vermeidung teurer Serialisierung großer Bild-Arrays. Bilddaten wandern nie durch Queues, sondern werden über `multiprocessing.shared_memory` in vom Betriebssystem verwaltete RAM-Blöcke geschrieben; die Queues transportieren nur kleine Proxy-Objekte mit Block-Referenz, Form und Datentyp. Ein Worker verbindet sich beim Zugriff mit dem Block und liest die Daten ohne Kopie (Zero-Copy). Weil das Betriebssystem den Speicher verwaltet, gilt ein striktes Ownership-Konzept: `close()` löst nur die lokale Bindung, `destroy()` gibt den Block global frei und darf genau einmal am Ende einer Inspektion aufgerufen werden. Ein zusätzlicher Worker-seitiger Garbage Collector zerstört Blöcke, auf die seit mehr als zehn Sekunden kein Zugriff erfolgte, um ein Volllaufen des flüchtigen Speichers zu verhindern.

Bild-Archiv und Shared-Memory-Blöcke liegen auf getrennten Pfaden, teilen sich physisch aber denselben Arbeitsspeicher: Das Bild-Archiv liegt in Produktion auf einer tmpfs-RAM-Disk, die Shared-Memory-Blöcke unter `/dev/shm`. Diese gemeinsame Ressource ist Ausgangspunkt von Case 2 (Abschnitt 5.2), das zehnstündige Freigabeintervall Gegenstand der Hypothese in Case 5 (Abschnitt 5.5).

3 Theoretische Grundlagen

3.1 Verfügbarkeit im Dauerbetrieb

Verfügbarkeit (*Availability*) ist eine zentrale Eigenschaft zuverlässiger Systeme. Avizienis et al. definieren sie als die Bereitschaft eines Systems, seinen spezifizierten Dienst korrekt zu erbringen [1]. Formal ergibt sie sich aus der mittleren Betriebsdauer zwischen Ausfällen (*Mean Time to Failure*, MTTF) und der mittleren Wiederherstellungszeit (*Mean Time to Repair*, MTTR):

$$A = \frac{MTTF}{MTTF + MTTR} \quad (1)$$

Gleichung 1 verdeutlicht, dass hohe Verfügbarkeit nicht allein durch seltene Ausfälle, sondern ebenso durch kurze Wiederherstellungszeiten erreicht wird. Für ein System ohne automatische Fehlererkennung ist die MTTR untrennbar an die Zeit gebunden, bis ein Mensch den Ausfall bemerkt. Während Entwicklungsumgebungen typischerweise täglich neu gestartet werden und Fehler durch manuelle Beobachtung sichtbar sind, akkumulieren sich im Dauerbetrieb Effekte, die über kurze Beobachtungsfenster unsichtbar bleiben. Ein System, das im Entwicklungsbetrieb stabil läuft, kann daher im 24/7-Betrieb durch eine andere Klasse von Fehlern ausfallen.

3.2 Fehlerklassifikation

Für diese Fallstudie sind zwei Verhaltensklassen bei Prozessausfällen relevant [2]. Bei einem *Fail-Stop*-Ausfall bricht ein fehlerhafter Prozess sofort ab und signalisiert seinen Ausfall unmissverständlich; der Fehler ist für das Gesamtsystem sichtbar und adressierbar. Bei einem *Fail-Silent*-Ausfall stellt der Prozess seine Funktion ohne Meldung ein, das Gesamtsystem läuft scheinbar weiter – ohne Alarm, ohne Neustart, ohne Log. Fail-Silent-Verhalten ist in industriellen Systemen besonders gefährlich, weil der Ausfall nicht durch technische Signale, sondern erst durch ausbleibende Produktionsergebnisse sichtbar wird.

Davon abzugrenzen sind *schleichende Fehler*, die nicht durch ein singuläres Ereignis, sondern durch die kontinuierliche Akkumulation eines Ressourcenverbrauchs entstehen. Charakteristisch ist die lange Zeitspanne zwischen Ursache und sichtbarem Symptom; im 24/7-Betrieb kann diese Stunden oder ganze Schichten betragen. Der Zustand ist besonders problematisch, wenn das System nach außen betriebsbereit wirkt, seinen eigentlichen Zweck aber nicht mehr erfüllt [3].

Ein *Kaskadeneffekt* liegt vor, wenn der Ausfall oder die Verlangsamung einer Komponente über gemeinsame Schnittstellen den Ausfall weiterer Komponenten verursacht [4]. Solche Effekte sind schwer zu diagnostizieren, da das sichtbare Symptom nicht die Ursache widerspiegelt: Das System kollabiert an einer anderen Stelle, als die ursprüngliche Schwachstelle liegt.

3.3 Prozesssupervision und das Watchdog-Pattern

In langlebigen Multiprocessing-Systemen ist es unzureichend, Prozesse nur beim Start zu überprüfen. Das *Supervisor-Modell* sieht einen dedizierten Überwachungsprozess vor, der den Laufzeitstatus aller untergeordneten Prozesse kontinuierlich überwacht und bei Ausfall automatisch reagiert – durch Neustart, Alarm oder geordnetes Herunterfahren. Dieses Konzept ist im Erlang/OTP-Framework als *Supervision Tree* etabliert: Dort überwachen Supervisoren ihre Kindprozesse und starten sie bei Absturz neu [5]. Das *Watchdog-Pattern* ist eine konkrete Ausprägung: Der Supervisor erwartet in regelmäßigen Abständen ein Lebenszeichen (*Heartbeat*) des überwachten Prozesses; bleibt es aus, gilt der Prozess als ausgefallen. Für eingebettete und industrielle Systeme sind mehrere Varianten dieses Musters mit unterschiedlichen Reaktionsstrategien dokumentiert [6].

Fehlt eine solche Logik, entstehen zwei Ausfallbilder. Im ersten hält ein abgestürzter Prozess, dessen Ausgabe von anderen Prozessen erwartet wird, das System in einem wartenden Zustand gefangen – ohne Timeout, ohne Alarm. Im zweiten läuft das System nach dem Ausfall eines Prozesses scheinbar weiter, Fehler werden erst durch ausbleibende Ausgaben sichtbar. Beide Szenarien erfordern manuelle Eingriffe, die im 24/7-Betrieb ohne Bereitschaftsdienst nicht zeitnah erfolgen.

3.4 Ressourcenerschöpfung und Backpressure

Eine *unbounded Queue* wächst ohne Obergrenze; bei anhaltender Produktionsrate und gedrossem Konsum akkumuliert sie Einträge, bis der Speicher erschöpft ist. Eine *bounded Queue* begrenzt das Wachstum, erzeugt dafür aber *Backpressure*: Produzenten, die in eine volle Queue schreiben wollen, werden blockiert. Kleppmann beschreibt Backpressure als notwendigen, aber häufig fehlerhaft implementierten Mechanismus. Ein blockierendes `put()` ohne Timeout überträgt den Stau auf alle vorgelagerten Komponenten [7]. In einem Multiprocessing-System mit gemeinsamer Queue kann so ein einzelner langsamer Konsument das gesamte System zum Einfrieren bringen.

Flüchtige Dateisysteme wie `tmpfs` und RAM-Disks liegen ausschließlich im Arbeitsspeicher [8]. Sie bieten hohe I/O-Geschwindigkeit, sind jedoch auf eine feste Größe begrenzt: Überschreitet das Schreibvolumen diese Grenze, schlagen Schreiboperationen mit `OSError: No space left on device` fehl. Eine RAM-Disk, die als Puffer zwischen schnellem Schreiben und langsamem Lesen dient, ist strukturell ein *Single Point of Failure* für alle abhängigen Komponenten.

3.5 Dev/Prod-Parity und Konfigurationsgefälle

Das Prinzip der *Dev/Prod-Parity* (Faktor X des *Twelve-Factor App*-Modells) fordert, Entwicklungs- und Produktionsumgebung so ähnlich wie möglich zu halten [9]. Abweichungen in Konfigurationsparametern können dazu führen, dass Fehler in der Entwicklung nicht reproduzierbar sind, die in Produktion deterministisch auftreten. Ein typisches Muster ist das Konfigurationsgefälle beim Verbindungs-Pooling: Entwicklungsumgebungen öffnen pro Anfrage eine frische Verbindung, Produktionsumgebungen nutzen aus Performance-Gründen persistente Verbindungen mit langer Lebensdauer. Die Fehlerklasse der veralteten Verbindung (*stale connection*) tritt damit im Entwicklungsbetrieb strukturell nicht auf. Fehlerbehandlungscode kann fehlen, ohne dass Tests darauf hinweisen; in Produktion führt das erste Auftreten der Fehlerbedingung zum unkontrollierten Absturz.

4 Methodik

4.1 Untersuchungsansatz

Die Untersuchung folgt dem Prinzip der *Fault Injection*: dem gezielten Einbringen definierter Fehlerzustände in ein laufendes System, um dessen Reaktion empirisch zu beobachten [10]. Für jeden Failure Case wird eine Hypothese über das erwartete Systemverhalten formuliert, ein reproduzierbares Fehlerszenario hergestellt und das tatsächliche Verhalten anhand definierter

Messgrößen erfasst. Wo ein Fehlerzustand ohne Eingriff in den Quellcode herstellbar ist, werden ausschließlich POSIX-Signale oder externe Werkzeuge eingesetzt; wo ein Eingriff nötig ist, wird dieser minimal gehalten und nach der Messung automatisch zurückgenommen.

4.2 Verfahren der Fehlerinjektion

Jeder Failure Case bildet einen realen Fehlerzustand durch ein möglichst direktes technisches Äquivalent nach. Tabelle 2 ordnet die eingesetzten Verfahren dem jeweils emulierten Fehler zu. Die ersten drei Verfahren kommen ohne jede Code-Änderung aus: Ein SIGKILL beendet einen Prozess hart und entspricht damit einem Absturz; SIGSTOP friert einen Prozess ein und bildet einen hängenden Aufruf nach, etwa einen blockierenden Datenbank-Call; pg_terminate_backend trennt eine Datenbankverbindung serverseitig. Nur zwei Cases erfordern einen minimalen, nach der Messung automatisch zurückgenommenen Eingriff: das gezielte Auslösen eines OSErrror(28) für die volle RAM-Disk sowie das Verschärfen von Timing-Parametern zur Provokation der vermuteten Race Condition.

Tabelle 2: Injektionsverfahren und emulierte Fehlerzustände

Verfahren	Emulierter Fehlerzustand	Case
SIGKILL	harter Prozessabsturz	1
OSErrror(28)	volle RAM-Disk (No space left)	2
SIGSTOP/SIGCONT	eingefrorener Prozess (hängender DB-Call)	3
pg_terminate_backend	serverseitig getrennte DB-Verbindung	4
Timing-Parameter	verschärftes Freigabe-Timing	5

4.3 Testumgebung

Alle Messungen erfolgen am Referenzsystem im Labor Bad Hersfeld. Das System wird über einen Simulator betrieben, der die Kamerahardware durch das Einspielen vorab erfasster Bilddaten ersetzt. Sämtliche Failure Cases sind dadurch ohne echte Kamerahardware unter kontrollierten Bedingungen reproduzierbar. Tabelle 3 fasst die relevanten Eigenschaften zusammen.

Tabelle 3: Eigenschaften der Testumgebung

Eigenschaft	Wert
Betriebssystem	Linux 6.17 (x86_64)
Arbeitsspeicher	64 GB
GPU	NVIDIA GeForce RTX 5090 (32 GB VRAM)
Laufzeitumgebung	Python 3.13
Startskript	main.py (mit geänderter Konfiguration für Simulation)
Bilddatenquelle	462 Bilder = 154 Inspektionen

4.4 Referenzmessung (Baseline)

Vor der Fehlerinjektion wurde eine Referenzmessung im Normalbetrieb ohne injizierte Fehler durchgeführt. Sie dient als Vergleichsbasis für alle Failure Cases. Das System startet neben dem Hauptprozess vier Service-Prozesse für `Logger`, `FrameGrabber`, `InspectionDataHandler` und `PipelineManager` sowie einen Pool von 12 Worker-Prozessen. Im stationären Zustand erreicht es einen Durchsatz von 1,74 Inspektionen pro Sekunde bei einer mittleren Pipeline-Laufzeit von 383 ms. Die zentrale Log-Queue bleibt dabei nahezu leer.

4.5 Messgrößen

Für jeden Case werden vergleichbare Messgrößen erhoben:

- *Time-to-Detection (TTD)*: Zeit von der Fehlerinjektion bis zur Erkennung durch das System (Shutdown oder Statusmeldung).
- *Folgeverhalten*: qualitative Beschreibung der Reaktion der übrigen Prozesse (Weiterlauf, Deadlock, Leerschleife, Absturz).
- *Queue-Tiefe und Speicherfüllstand über die Zeit*: quantitative Erfassung schleichender und kaskadierender Effekte.
- *Graceful Degradation*: ob das System bei einem Teilausfall in einen kontrollierten Zustand übergeht oder vollständig ausfällt.

4.6 Grenzen der Untersuchung

Die Ergebnisse sind vor dem Hintergrund mehrerer methodischer Einschränkungen zu lesen. Erstens erfolgen alle Messungen am Referenzsystem im Labor, nicht auf der späteren Kundenhardware; die absoluten Zeiten sind daher nur eingeschränkt übertragbar, während die zugrunde liegenden Fehlermechanismen von der Hardware unabhängig sind. Zweitens ersetzt der Simulator die Kamerahardware: Hardwarespezifische Fehler wie ein Kamera-Disconnect werden nicht direkt erzeugt, ihr Effekt auf das System aber über den Ausfall des `FrameGrabbers` abgebildet. Drittens sind die Beobachtungsfenster mit 20 bis 120 Sekunden bewusst kurz gehalten; schleichende Effekte über ganze Schichten werden aus den gemessenen Raten hochgerechnet, nicht über die volle Dauer beobachtet. Viertens ist Case 4 aufgrund der maskierenden Entwicklungskonfiguration analytisch statt empirisch belegt. Schließlich stützt sich jeder Case auf Einzelmessungen; der Fokus liegt auf den qualitativen Fehlermechanismen und ihren Größenordnungen, nicht auf statistisch abgesicherten Verteilungen.

5 Failure Cases

Dieses Kapitel stellt die fünf untersuchten Failure Cases dar. Jeder Case erläutert den relevanten Systemkontext und die zugrunde liegende Schwachstelle, beschreibt das gemessene Verhalten und leitet daraus Interpretation und Handlungsempfehlung ab.

5.1 Case 1 – Lücken in der Laufzeit-Prozesssupervision

Im stationären Betrieb überwacht das System ausschließlich den `PipelineManager` über einen Poll im Sekundentakt. Die drei weiteren Service-Prozesse für Frame-Grabbing, Datenbereitstellung und Logging laufen nach erfolgreichem Start ohne jede Laufzeitüberwachung; eine Supervisor-Logik im Sinne von Abschnitt 3.3 existiert nicht. Um die Auswirkung dieser Lücke zu quantifizieren, wurde jeder Service-Prozess einzeln per `SIGKILL` zum Absturz gebracht. Tabelle 4 zeigt, ob der Hauptprozess den Ausfall erkennt und welche Systemreaktion daraus folgt.

Tabelle 4: Erkennung von Service-Prozess-Ausfällen

Zielprozess	Erkannt	TTD	Systemreaktion
PipelineManager	ja	0,992 s	geordneter Shutdown aller Subsysteme
FrameGrabber	nein	–	Deadlock in Pipeline-Schritt 1
InspDataHandler	nein	–	stiller Ausfall, laufende Inspektionen enden
Logger	nein	–	stiller Ausfall, kein Log-Output mehr

Die Tabelle macht die zentrale Schwachstelle sichtbar: Nur der Prozess, der in der Hauptschleife abgefragt wird, löst eine Reaktion aus. Alle anderen Ausfälle bleiben im Beobachtungsfenster ohne technische Meldung. Beim `FrameGrabber` ist die Folge ein Pipeline-Stillstand in Schritt 1, weil keine neuen Bilddaten eintreffen. Beim `InspectionDataHandler` und beim `Logger` läuft das System zunächst weiter, verliert aber Datenbereitstellung beziehungsweise Protokollierung. Die Ausfälle entsprechen damit dem Fail-Silent-Verhalten [2]. Abhilfe schafft eine Überwachung aller Service-Prozesse oder eine externe Supervision, etwa über eine `systemd`-Unit mit `Restart=on-failure`.

5.2 Case 2 – Flüchtiger Bildspeicher als Single Point of Failure

Verarbeitete Bilder werden auf einer als `tmpfs`-RAM-Disk gemounteten Partition abgelegt. Die zuständige Funktion `store_images_to_disk` enthält keine Fehlerbehandlung für `OSError`, und der Füllstand wird nicht überwacht. Da eine RAM-Disk auf eine feste Größe begrenzt ist (Abschnitt 3.4), schlagen Schreiboperationen bei Erschöpfung fehl. Im Normalbetrieb wächst das Bildverzeichnis um rund 21,8 MB pro Inspektion, was bei Produktionsdurchsatz einer Schreibrate von etwa 38 MB/s entspricht. Bei 12 GB freiem Speicher wäre die RAM-Disk

damit nach ungefähr fünf Minuten erschöpft, sofern keine Rotation oder Auslagerung greift. Abbildung 2 ordnet die Messung ein: Schon der kurze Referenzlauf zeigt lineares Wachstum des Bildverzeichnisses; ohne Begrenzung wird daraus im Dauerbetrieb ein Ausfallpunkt. Zur Analyse des Fehlerfalls wurde eine volle Disk durch das gezielte Auslösen eines `OSError` (28) nachgebildet (Tabelle 5).

Tabelle 5: Verhalten bei erschöpftem Bildspeicher

Messgröße	Wert
Schreibrate (Hochrechnung Produktionslast)	$\approx 38 \text{ MB/s}$
Zeit von <code>OSError</code> bis PM-Absturz	35,2 s
PM-Absturz bis Shutdown	2,1 s
Graceful Degradation	nein

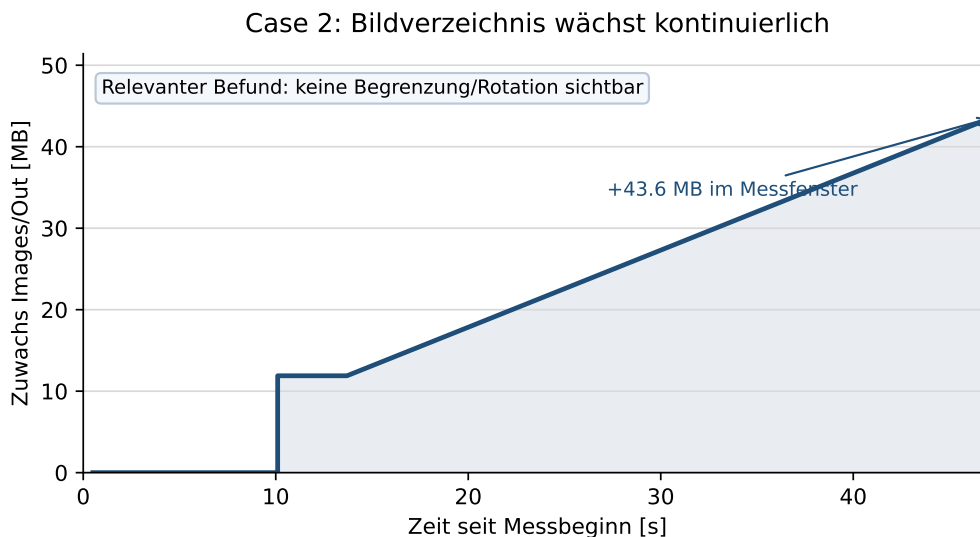


Abbildung 2: Wachstum des Bildverzeichnisses `Images/Out` im Referenzlauf. Der lineare Anstieg ist der relevante Befund; in Produktion liegt dieser Pfad auf einer begrenzten RAM-Disk.

Quelle: Eigene Darstellung auf Basis eigener Messdaten

Ein einzelner fehlgeschlagener Schreibvorgang beendet den produktiven Ablauf. Zwischen dem Fehler und dem erkannten Absturz liegt eine Stillstandsphase von gut 35 Sekunden, in der der `PipelineManager` auf das Ergebnis des abgestürzten `Workers` wartet. Die RAM-Disk ist damit ein einzelner Ausfallpunkt ohne Rückfallebene. Shared-Memory-Blöcke und Bildarchiv liegen zwar auf unterschiedlichen Pfaden, nutzen aber denselben physischen Arbeitsspeicher. Eine Fehlerbehandlung in `store_images_to_disk`, die bei vollem Bildspeicher nur die Archivierung überspringt, sowie ein Füllstands-Monitoring mit Alarm unterhalb von 20 % freiem Speicher würden diesen Ausfall verhindern.

5.3 Case 3 – Log-Queue-Stau als Pipeline-Blocker

Alle Worker und der PipelineManager schreiben Log-Einträge synchron über `log_queue.put()` in eine zentrale, auf 1000 Einträge begrenzte Queue. Der Aufruf erfolgt blockierend ohne Timeout, und der Zustand `queue.Full` wird nicht abgefangen – eine Konstruktion, die Backpressure im Sinne von Abschnitt 3.4 erzeugt. Um eine hängende Datenbankschicht nachzubilden, wurde der Logger per SIGSTOP eingefroren (Tabelle 6).

Tabelle 6: Verhalten bei eingefrorenem Logger-Prozess

Messgröße	Wert
Durchsatz vor Eingriff	17 Pipelines / 5 s
Pipelines nach SIGSTOP	5 (bereits in Bearbeitung)
Zeit bis vollständiger Stillstand	≈ 1,3 s
Stillstand im Beobachtungsfenster	27,7 s, kein Alarm
Queue nach SIGCONT leer	< 0,3 s

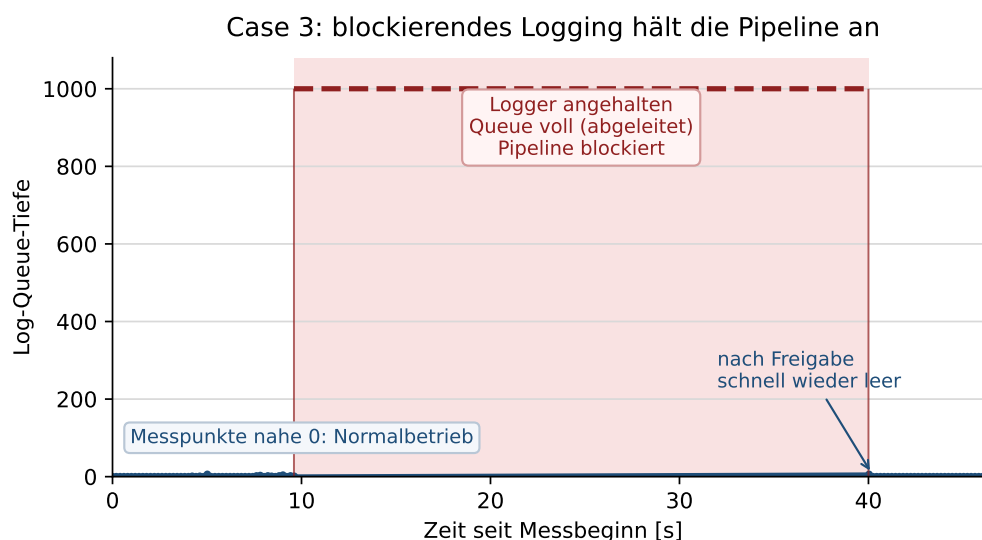


Abbildung 3: Log-Queue bei angehaltenem Logger. Die schattierte Phase enthält keine Messpunkte, weil der Logger-Prozess samt Mess-Thread pausiert. Aus dem anschließenden Rücksprung lässt sich ableiten, dass die Queue in dieser Phase vollgelaufen ist.

Quelle: Eigene Darstellung auf Basis eigener Messdaten

Im Normalbetrieb bleibt die Queue nahezu leer. Wird der Logger angehalten, erreicht die Queue ihre Kapazitätsgrenze; danach blockieren alle Prozesse, die beim nächsten Log-Aufruf in diese Queue schreiben wollen. Der Stillstand entsteht also nicht in der Bildverarbeitung selbst, sondern in einer gemeinsam genutzten Hilfskomponente. Nach Fortsetzung des Loggers leert sich die Queue in unter 0,3 Sekunden. Der Engpass ist damit nicht die normale Schreibgeschwindigkeit, sondern das blockierende Verhalten im Fehlerfall. Eine Umstellung auf `put(timeout=0.1)` mit Drop-on-Full entkoppelt Logging und Pipeline: einzelne Log-Einträge gehen verloren, die Verarbeitung bleibt jedoch lauffähig.

5.4 Case 4 – Veraltete Datenbankverbindung nach Produktionspause

Die Persistierung der Inspektionsergebnisse erfolgt über das Django-ORM in der Funktion `convert_inspection_to_model`, die weder einen `OperationalError` abfängt noch `close_old_connections()` aufruft. Entscheidend ist ein Konfigurationsgefälle (Abschnitt 3.5): Während die Entwicklung mit `CONN_MAX_AGE = 0` pro Aufruf eine frische Verbindung öffnet, hält die Produktion mit `CONN_MAX_AGE = 60` eine persistente Verbindung. Zur Prüfung wurden in der Entwicklungsumgebung während des Betriebs alle Datenbankverbindungen über `pg_terminate_backend` getrennt. Dieser Versuch ist kein direkter Produktions-Crash-Test, sondern eine Negativkontrolle: Mit `CONN_MAX_AGE = 0` öffnet Django pro Datenbankzugriff eine neue Verbindung, sodass keine veraltete Verbindung entstehen kann. Vor dem Eingriff waren rund 27 Inspektionen verarbeitet; 27 Datenbankverbindungen wurden terminiert. Das System verarbeitete weiter, ein `OperationalError` trat nicht auf.

Das Risiko ergibt sich aus der Produktionskonfiguration. Nach einer Pause trennt PostgreSQL oder eine vorgelagerte Firewall die inaktive Verbindung; der erste Schreibzugriff danach trifft auf eine für gültig gehaltene, tatsächlich tote Verbindung und löst einen `OperationalError` aus. Da dieser Fehler nicht behandelt wird, kann der Worker abstürzen und über den bereits beschriebenen Mechanismus den `PipelineManager` beenden. Der Fehler tritt damit beim ersten Materialdurchlauf nach einer Produktionspause auf und wird durch die Entwicklungskonfiguration maskiert. Ein Aufruf von `close_old_connections()` vor dem Schreibzugriff oder das Setzen von `CONN_MAX_AGE = 0` auch in Produktion behebt das Problem; die zusätzliche Latenz ist bei lokaler Datenbank vernachlässigbar [11].

5.5 Case 5 – Vermutete Race Condition im Speichermanagement

Worker-Prozesse geben ihre Shared-Memory-Blöcke über eine Routine frei, die Blöcke nach mehr als zehn Sekunden ohne Zugriff zerstört [12]. Die Ausgangshypothese lautete, dass diese Freigabe unter Last Blöcke zerstören könnte, die der `PipelineManager` noch benötigt, und so einen `FileNotFoundError` auslöst. Um den vermuteten Konflikt zu provozieren, wurde die Freigabeschwelle von zehn auf zwei Sekunden gesenkt und ein nachgelagerter Verarbeitungsschritt gezielt um vier Sekunden verzögert. Über 120 Sekunden Laufzeit trat jedoch weder ein `FileNotFoundError` noch ein `BrokenPipeError` oder ein Absturz auf.

Die Hypothese wird durch zwei Mechanismen widerlegt. Erstens greift der letzte lesende Schritt typischerweise nach 45 bis 107 Millisekunden auf die Blöcke zu, während die Freigabe frühestens nach zehn Sekunden erfolgt. Zweitens ist die Freigabeoperation idempotent: Ein zweiter Aufruf liefert nur einen negativen Status zurück. Der vermutete Pfad über einen

langsamen Persistierungsschritt entfällt, weil dieser Schritt nicht auf Shared-Memory-Blöcke zugreift. Für diesen Case besteht daher kein Sofortbedarf. Sinnvoll bleiben ein Monitoring der Step-5-Laufzeit und langfristig ein Referenzzähler für Speicherblöcke.

6 Fazit und Handlungsempfehlungen

Die Fallstudie hat fünf Failure Cases des ROSI-Systems auf Prozess- und Betriebssystemebene untersucht und damit die drei eingangs formulierten Forschungsfragen beantwortet.

Zu FF2 (Systembeobachtbarkeit) zeigt Case 1 die größte Lücke: Drei von vier Service-Prozessen fallen ohne Erkennung aus. Da das System über kein externes Alerting verfügt, bleibt die Time-to-Detection in diesen Fällen unbegrenzt. Das erhöht die Wiederherstellungszeit und senkt nach Gleichung 1 die Verfügbarkeit. Zu FF1 (Ressourcenverwaltung) belegt Case 2 die fehlende Rückfallebene des flüchtigen Bildspeichers, während Case 5 eine angenommene Race Condition widerlegt. Zu FF3 (Datenpersistenz und Kaskaden) zeigen Case 3 und Case 4 zwei unterschiedliche Mechanismen: einen systemweiten Stillstand durch Backpressure sowie ein durch die Entwicklungskonfiguration maskiertes Verbindungsrisiko.

6.1 Übergreifende Befunde

Zwei Muster ziehen sich durch die Ergebnisse. Erstens fehlt eine durchgängige *Graceful Degradation*: Ein lokaler Fehler – ein abgestürzter Prozess, ein fehlgeschlagener Schreibvorgang, eine tote Verbindung – führt zum Ausfall des Gesamtsystems statt zu einem kontrollierten Teilbetrieb. Zweitens bestehen *blinde Flecken im Monitoring*: Weder Queue-Tiefen noch Speicherfüllstände noch der Zustand der Service-Prozesse werden zur Laufzeit überwacht. Besonders Case 3 zeigt, dass eine verlangsamte Hilfskomponente über eine gemeinsame Queue die eigentliche Pipeline blockieren kann.

Beide Muster wirken über die MTTR auf die Verfügbarkeit. Solange ein Ausfall erst durch ausbleibende Inspektionsergebnisse oder manuelle Beobachtung auffällt, ist die Time-to-Detection an die Anwesenheit von Personal gebunden und kann im 24/7-Betrieb ohne Bereitschaftsdienst Stunden betragen. Nach Gleichung 1 senkt jede Verlängerung der MTTR die Verfügbarkeit unmittelbar – unabhängig davon, wie selten der Fehler auftritt. Eine automatische Erkennung, die die Time-to-Detection von Stunden auf Sekunden verkürzt, verbessert die Verfügbarkeit damit selbst dann deutlich, wenn die Ausfallrate unverändert bleibt. Genau hier setzen die folgenden Empfehlungen an.

6.2 Priorisierte Handlungsempfehlungen

Tabelle 7 fasst die abgeleiteten Maßnahmen zusammen. Die ersten vier Maßnahmen verbessern direkt die Verfügbarkeit: Sie verkürzen entweder die Time-to-Detection oder entkoppeln einen lokalen Fehler vom Gesamtsystem. Case 5 erfordert keinen Sofortfix, eignet sich aber als Anlass für Step-5-Monitoring.

Tabelle 7: Priorisierte Handlungsempfehlungen

Case	Aufwand	Maßnahme
1	gering	Supervision aller Service-Prozesse oder <code>systemd</code> -Unit
2	gering	Fehlerbehandlung in der Bildspeicherung + Füllstands-Monitoring
3	minimal	<code>put(timeout)</code> mit Drop-on-Full in der Log-Queue
4	minimal	<code>close_old_connections()</code> bzw. <code>CONN_MAX_AGE = 0</code>
5	–	kein Sofortfix; optional Referenzzähler und Step-5-Monitoring

6.3 Ausblick

Die wichtigsten Hebel für die Produktionsreife liegen in einer durchgängigen Prozessüberwachung sowie einem Monitoring der bislang unbeobachteten Ressourcen. Beide Maßnahmen adressieren nicht nur einzelne Cases, sondern die übergreifenden Muster fehlender Beobachtbarkeit und fehlender Graceful Degradation. Eine Übertragung der Untersuchung auf das spätere Kundensystem – mit abweichender Hardware und aktiver Produktionskonfiguration – bleibt offen und würde insbesondere das in Case 4 analytisch hergeleitete Verbindungsrisiko empirisch absichern.

Literaturverzeichnis

- [1] A. Avizienis, J.-C. Laprie, B. Randell und C. Landwehr, „Basic Concepts and Taxonomy of Dependable and Secure Computing“, *IEEE Transactions on Dependable and Secure Computing*, Jg. 1, Nr. 1, S. 11–33, 2004. DOI: 10.1109/TDSC.2004.2
- [2] F. Cristian, „Understanding fault-tolerant distributed systems“, *Communications of the ACM*, Jg. 34, Nr. 2, S. 56–78, 1991. DOI: 10.1145/102792.102801
- [3] K. P. Birman, *Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services*. Springer, 2012, ISBN: 978-1-4471-2415-3.
- [4] H. Qi, S. Bhatt und A. Bhatt, „System Reliability Under Cascading Failure Models“, *IEEE Transactions on Reliability*, 2016. DOI: 10.1109/TR.2015.2500283
- [5] J. Armstrong, *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf, 2007, ISBN: 978-1-934356-00-5.
- [6] M. J. Pont und R. H. L. Ong, *Using watchdog timers to improve the reliability of TTCS embedded systems: Seven new patterns and a case study*, VikingPLoP 2002. [Online]. Available: <https://www.cs.le.ac.uk/SoftSD/resources/viking02.pdf>, Zuletzt abgerufen: 1. Juni 2026, 2002.
- [7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [8] The Linux Kernel Organization, *tmpfs – a virtual memory filesystem*, Linux Kernel Documentation. [Online]. Available: <https://www.kernel.org/doc/html/latest/filesystems/tmpfs.html>, Zuletzt abgerufen: 30. Mai 2026.
- [9] A. Wiggins, *The Twelve-Factor App*, [Online]. Available: <https://www.12factor.net/>, Zuletzt abgerufen: 1. Juni 2026, 2011.
- [10] M.-C. Hsueh, T. K. Tsai und R. K. Iyer, „Fault injection techniques and tools“, *IEEE Computer*, Jg. 30, Nr. 4, S. 75–82, 1997. DOI: 10.1109/2.585157
- [11] Django Software Foundation, *Databases – Persistent connections*, Django Documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/ref/databases/#persistent-connections>, Zuletzt abgerufen: 30. Mai 2026.
- [12] Python Software Foundation, *multiprocessing.shared_memory – Shared memory for direct access across processes*, Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.shared_memory.html, Zuletzt abgerufen: 30. Mai 2026.

Eidesstattliche Erklärung

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Ich erkläre weiterhin, dass die vorliegende Arbeit in gleicher oder ähnlicher Form noch nicht im Rahmen eines anderen Prüfungsverfahrens eingereicht wurde.

Bad Hersfeld, den 24. Juli 2026

Luca Michael Schmidt